

28th Jan 2021 | DA6401

- Network of neurons in MLP can represent non-linear decision boundaries.
- Vol. of space where the config. is present becomes ∞ small as compared to large volume.

→ problem of parameter space explosion.

→ Stochastic Wt. guessing → network is black box. Randomly sample wt. vectors w from high dim. space until convergence.

→ brute-force isn't a viable solution

→ Parameterizing → as a funcⁿ of learnable params → w . Goal is to find the optimal value of θ^* .

model prediction $\hat{y} = f(x; w)$
 network input x all the wts. & biases w

$$D = \{x^i, y^i\}_{i=1}^N$$

measure the loss

MSE [mean Sq. Error]

measure the discrepancy btw. $\hat{y}^i$ & y^i

$$J(w) = \frac{1}{N} \sum_{i=1}^N (y^i - \hat{y}^i)^2$$

→ for a fixed w ,

$$J(w) : \mathbb{R}^d \rightarrow \mathbb{R}$$

$N \rightarrow$ total no. of data pts.

loss landscape

- coordinates → value of wts. $\{w_1, \dots, w_d\}$
- altitude → loss value $J(w)$

→ this is differentiable, provides the direcⁿ of steepest ascent.

→ gradient → vector of partial derivatives w.r.t all wts.

$$\nabla_w J = \begin{bmatrix} \frac{\partial J}{\partial w_1} & \frac{\partial J}{\partial w_2} & \dots & \frac{\partial J}{\partial w_n} \end{bmatrix}^T$$

→ -ve → $-\nabla_w J$, pts. in the direcⁿ of steepest descent.

Imp. → Proof of how $-\nabla_w J$ using Taylor Series expansion.

⇒ $J(w) \rightarrow$ scalar valued funcⁿ $w \in \mathbb{R}^n$ $\nabla_w J(w) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_n} \end{bmatrix}$

① 1st order Taylor Series expansion

$\delta \in \mathbb{R}^n$

$$J(w + \delta) \approx J(w) + \nabla J(w)^T \delta$$

② Δ in the funcⁿ value

$$\Delta J = J(w + \delta) - J(w) \approx \nabla J(w)^T \delta$$

$$\min \Delta J \Rightarrow \min \nabla J^T \delta$$

③ constraints on the step size

$$\|\delta\| = \epsilon$$

$$\nabla J^T \delta = \|\nabla J\| \|\delta\| \cos \theta = \epsilon \|\nabla J\| \cos \theta$$

④ the direcⁿ that minimises ΔJ

$$\text{since } \epsilon \|\nabla J\| > 0, \min \Delta J \Rightarrow \min \cos \theta$$

$$\cos \theta = -1 \Rightarrow \theta = \pi$$

⇒ δ pts. opposite to ∇J

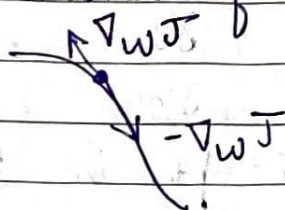
⑤ direction of steepest descent

$$\delta = -\epsilon \frac{\nabla J}{\|\nabla J\|}$$

$$\|\nabla J\|$$

$$w_{\text{new}} = w - \alpha \nabla J(w)$$

- move opp. to the direcⁿ of gradient to get to the minima of the loss funcⁿ $\rightarrow$ GD

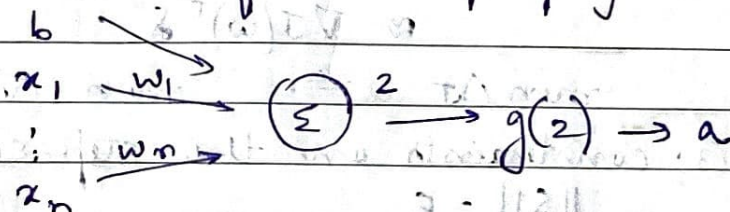


it is a learning algo.

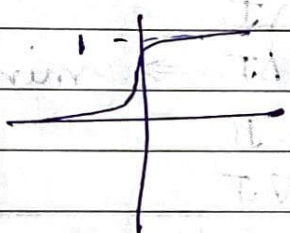
- any ML (supervised) has $\rightarrow$ data (D), model (f), params (e.g. w), Obj. (J(w)), Learning Algo. (any optimizaⁿ method)
- Perceptron limitaⁿ $\rightarrow$ capacity, Threshold (fixed)
- Modern neuron $\rightarrow$

LC (Linear) $\rightarrow z = w \cdot x + b$

diff. actuaⁿ $\rightarrow a = g(z)$
 gradient. It is used in adjusting wts. to fix this errors. (see for backpropagaⁿ).



Sigmoid Neuron $\rightarrow \sigma(z) = \frac{1}{1 + e^{-z}}$



smoother $z = w \cdot x + b$

- $\rightarrow$ continuous output
- $\rightarrow$ probabilistic interpretation $P(y=1|x)$
- $\rightarrow$ differentiable $\rightarrow$ slope exists everywhere

$\sigma'(z) \neq 0$ (mostly)

//_

→ Find the derivative of $\sigma(z)$ w.r.t input?

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad z = w^T x + b$$

$$\Rightarrow \sigma(z) = (1+e^{-z})^{-1}$$

derivative w.r.t. z & chain rule

$$\begin{aligned} \frac{d\sigma}{dz} &= -1 \cdot (1+e^{-z})^{-2} \cdot \frac{d}{dz}(1+e^{-z}) \\ &= -(1+e^{-z})^{-2} \cdot (-e^{-z}) \end{aligned}$$

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$

--- (1)

But, $\sigma(z) = \frac{1}{1+e^{-z}}$ so,

$$1 - \sigma(z) = \frac{e^{-z}}{(1+e^{-z})}$$

$$\sigma(z)(1 - \sigma(z)) = \frac{e^{-z}}{(1+e^{-z})^2}$$

--- (2)

so, 1 & 2 eqⁿ matches.

$$\boxed{\left(\frac{d\sigma(z)}{dz} \right) = \sigma(z)(1 - \sigma(z))}$$

$$\frac{\partial \sigma}{\partial x} = \frac{d\sigma}{dz} \times \frac{dz}{dx}$$

$\Rightarrow$ w.r.t input features

$$z = w^T x + b \Rightarrow \frac{dz}{dx} = w$$

$$\boxed{\frac{\partial \sigma}{\partial x} = \sigma(z)(1 - \sigma(z)) w}$$